Redis Fundamentals

data structures, one fast thread, persistence and eviction

Watch the 5-min video

In one line: Redis is an in-memory data structure server (strings, hashes, lists, sets, sorted sets, streams); it's fast on one thread because data is in memory, commands are simple, there are no locks and an event loop serves many connections; every command is atomic, Lua scripts make multi-step logic atomic, RDB and AOF give persistence, and maxmemory policies decide eviction.

1 One core, hundreds of thousands of ops per second

Redis powers caches, leaderboards, rate limiters, session stores and queues, and one instance handles hundreds of thousands of operations per second on a single core. But it's single-threaded, so how is it so fast? And if it lives in memory, what happens when it restarts?

2 Data structures, not just strings

Redis is an in-memory **data structure server**. Choosing the right structure often turns a complex feature into one command.

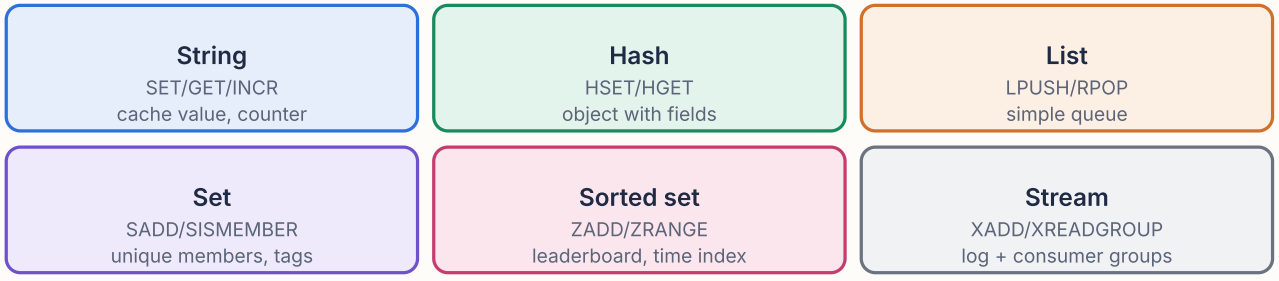


Figure 1. The core Redis data types, a typical command for each, and what they're used for.

3 Why it's fast on one thread

- **Everything is in memory**, so there's no disk wait on reads.
- **Simple commands**: most run in O(1) or O(log n).
- **One thread**: no locks, no context switches between threads.
- **I/O multiplexing**: an event loop (epoll on Linux) handles thousands of connections at once.

The network is usually the bottleneck, not the CPU. Since Redis 6, network I/O can use extra threads (`io-threads`), while commands still run on one.

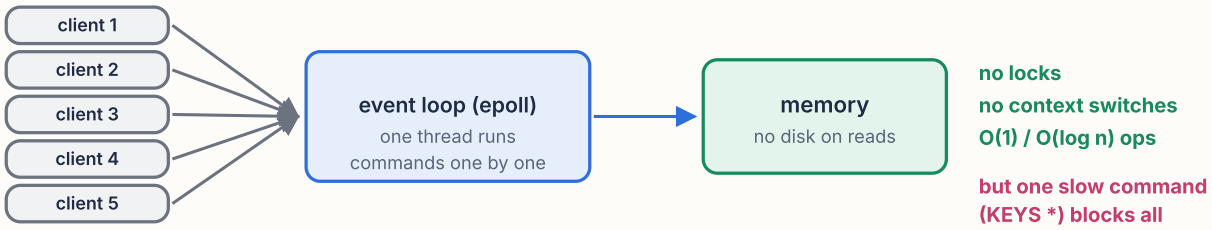


Figure 2. Many connections, one event loop, data in memory: fast, but one slow command delays everyone.

4 Atomic by design

Because commands run one at a time, **every command is atomic**: `INCR` can't be interrupted halfway. For multi-step logic, a **Lua script** (`EVAL`) runs atomically on the server: no other command runs in between. That's how a rate limiter checks and updates a counter safely. `MULTI` / `EXEC` transactions also run a batch without interleaving, but can't branch on results.

The flip side: one slow command blocks everyone. `KEYS *` on millions of keys, `HGETALL` on a huge hash or a long Lua script freezes the server; use `SCAN` instead.

5 Persistence: RDB and AOF

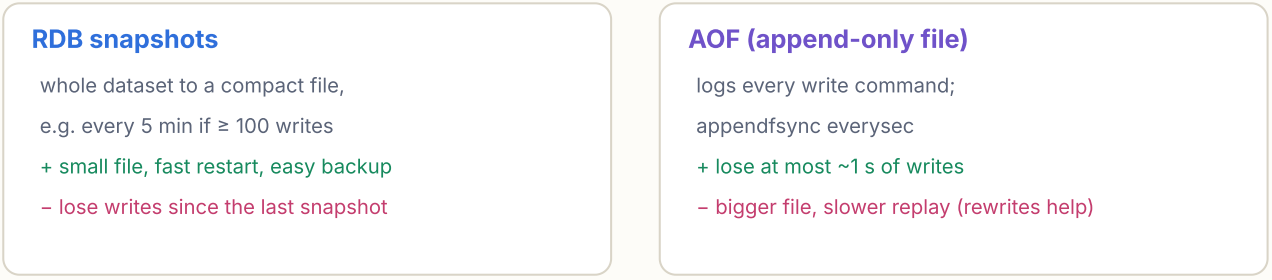


Figure 3. The two persistence options. Many deployments use both: AOF for durability, RDB for backups and fast restarts.

6 When memory fills up: eviction policies

Policy	Behaviour	Use for
`noeviction`	reject new writes with an error	Redis as a primary data store or queue
`allkeys-lru` / `allkeys-lfu`	evict from all keys (lessons 20, 21)	a pure cache: the usual choice
`volatile-lru` / `-lfu` / `-ttl`	evict only keys that have a TTL	mixed: cache keys with TTLs + keys that must stay

Set `maxmemory` and a policy explicitly, so behaviour under pressure is predictable.

WORKED EXAMPLE: A LEADERBOARD AND A RATE LIMITER

Leaderboard as a sorted set: `ZADD lb 1520 alice` sets a score, `ZREVRANGE lb 0 9 WITHSCORES` gets the top ten, `ZREVRANK lb alice` gives a player's position, each in O(log n). **Rate limiter**: `INCR rate:user42:202611061402`, then `EXPIRE … 60` on the first hit, both in one Lua script so increment and check happen atomically; over 100 → reject. And **pub/sub** can broadcast events, like cache invalidations, to many subscribers.

COMMON MISTAKES

- `KEYS *` in production code.
- No `maxmemory`, so the OS kills Redis when memory runs out.
- Treating Redis as durable by default: without AOF, a restart loses everything since the last snapshot.
- Huge values or collections (a 10 million element list) that make single commands slow.

7 Key terms

Sorted set (ZSET): members ordered by a score, O(log n) ops

Stream: append-only log with consumer groups

Event loop: one thread serving many sockets via epoll

Lua script: server-side script that runs atomically

RDB: periodic snapshot file

AOF: log of every write command

maxmemory policy: what to do when memory is full

SCAN: incremental key iteration, unlike blocking KEYS

8 Interview questions

Q1. Why is Redis fast despite being single-threaded? BASIC

Data lives in memory, commands are simple and efficient, a single thread avoids locks and context switches, and an event loop with I/O multiplexing serves many connections.

Q2. RDB versus AOF? BASIC

RDB takes periodic compact snapshots that restart fast but can lose minutes of writes, while AOF logs every write and loses at most about a second, at the cost of bigger files and slower restarts.

Q3. Which eviction policy for a cache? BASIC

allkeys-lru or allkeys-lfu with an explicit maxmemory; noeviction is for when Redis holds data you must not lose.

Q4. Build a top-10 leaderboard for a game with millions of players. SCENARIO

A sorted set keyed by game: ZINCRBY on each score change, ZREVRANGE 0 9 for the top ten and ZREVRANK for a player's rank, all O(log n); shard by game or season if one set gets too big.

Q5. Redis latency spikes every few minutes. What could it be? SCENARIO

A slow command (KEYS, big HGETALL, long Lua), RDB fork on a large dataset, AOF fsync on a slow disk, or eviction storms; check SLOWLOG, LATENCY DOCTOR and persistence settings.

Q6. How do you make 'check then update' atomic in Redis? TRICKY

Use a single command if one exists (INCR, SET NX), or a Lua script that reads and writes in one server-side step; WATCH with MULTI/EXEC gives optimistic locking as an alternative.

Next: SD-F-014 · Redis Replication, Sentinel and Cluster backendroadmap.com